

The Event System

A plain-English guide for developers coming from WordPress hooks & actions.

Gated Demand Architecture (GDA) · Domain Events & Integration Events · Generated 2026-06-22

1. Start Here: The WordPress Mental Model

In WordPress you wire features together with **hooks**. Something happens, you *fire* a signal, and other code that was *listening* reacts:

```
// WordPress
do_action( 'user_registered', $user_id );           // fire the signal

add_action( 'user_registered', 'send_welcome_email' ); // listen & react
function send_welcome_email( $user_id ) { /* ... */ }
```

The PSP platform works on the **exact same idea** — "something happened, others react" — but it makes two things explicit that WordPress leaves loose:

1. **Who is allowed to listen** (declared up front, not discovered at runtime).
2. **When it is safe to fire** (only after the database change is truly saved).

One-line translation: `do_action()` $\approx$ `$eventBus->dispatch($event)` , and `add_action()` $\approx$ `$events->subscribe('event.name', Listener::class)` . The big difference: PSP events are **typed objects**, not loose strings + arguments.

2. The One Big Difference: Two Kinds of Events

WordPress has one kind of hook. PSP has **two**, and picking the right one is the whole game. Think of it as the difference between a *private note to yourself* and a *public announcement to the whole building*.

	Domain Event (private note)	Integration Event (public announcement)
Who hears it	Only code <i>inside the same module</i>	Any other module that subscribes
When it fires	<i>During</i> the database transaction	<i>After</i> the transaction is committed
If the save fails	Thrown away — never happened	Never fired in the first place
What it carries	Rich domain objects (Money, InvoiceId)	Plain values only (string, int, float)
Lives in folder	Domain/Events/	API/IntegrationEvents/
Fired with	<code>\$collector->collect(\$e)</code>	<code>\$eventBus->dispatch(\$e)</code>
WordPress analogy	An internal helper function call	<code>do_action()</code> across plugins

The #1 rule that prevents bugs: never announce something to other modules (Integration Event) *before* you are 100% sure it was saved. If you email a customer "your invoice is ready" and then the database save rolls back, you have lied to them. That is exactly the bug the two-event split is designed to make impossible.

3. Domain Events — The Private Note

A domain event records "this happened" *inside* a single business module. It is a small, immutable object named in the **past tense**.

```
// Domain/Events/InvoiceCreatedDomainEvent.php
final readonly class InvoiceCreatedDomainEvent implements DomainEventContract
{
    public function __construct(
        public InvoiceId      $invoiceId,
        public Money          $total,          // rich domain objects are fine here
        public \DateTimeImmutable $occurredAt,
    ) {}
}
```

How it flows

The entity (e.g. an **Invoice**) *records* events as things happen to it. The service then *collects* them into a buffer that is tied to the database transaction:

```
// inside the Entity – just remembers what happened
public static function create(...): self {
    $inv = new self(...);
    $inv->domainEvents[] = new InvoiceCreatedDomainEvent($inv->id, $inv->total, ...)
    return $inv;
}

// inside the Service – the mandatory transaction shape
$this->collector->beginCollection();    // open the buffer
$this->transaction->begin();
try {
    $invoice = Invoice::create(...);
    foreach ($invoice->releaseEvents() as $e) {
        $this->collector->collect($e);    // buffer the events
    }
    $this->repository->save($invoice);
    $this->transaction->commit();          // SAVED for real
} catch (\Throwable $e) {
    $this->transaction->rollback();
    $this->collector->discard();           // throw the events away – no phantoms
    throw $e;
}
```

Why the buffer? The `DomainEventCollector` holds events in memory *during* the transaction. If the save works, they are released. If it rolls back, `discard()` wipes them. This is why a half-finished operation can never leak a "created" event. WordPress has no equivalent safety net — a `do_action` fires immediately whether or not your `$wpdb` insert later fails.

4. Integration Events — The Public Announcement

This is the one that behaves most like a WordPress `do_action()` across plugins. It tells *other* modules that something is officially done. Two hard rules: it is **versioned**, and it carries **plain values only** (because other modules do not have your domain objects).

```
// API/IntegrationEvents/TaskCreatedIntegrationEvent.php (real plugin example)
final readonly class TaskCreatedIntegrationEvent implements IntegrationEventContract
{
    public string $version;

    public function __construct(
        public string $taskId,
        public string $title,
        public string $occurredAt,
    ) { $this->version = '1.0'; }

    public function name(): string { return 'task.created'; } // the "hook name"
    public function version(): string { return $this->version; }
    public function payload(): array { // the "hook args"
        return ['taskId' => $this->taskId, 'title' => $this->title,
            'occurredAt' => $this->occurredAt, 'version' => $this->version];
    }
}
```

It is dispatched **after** the commit — outside the `try/catch`:

```
$this->transaction->commit(); // only past here on success
// ...
$this->eventBus->dispatch(new TaskCreatedIntegrationEvent(
    taskId: $task->id()->value(),
    title: $task->title(),
    occurredAt: (new \DateTimeImmutable())->format(DATE_RFC3339),
));
```

5. Listening to an Event (the `add_action` part)

This is a two-step process. In WordPress you just call `add_action` anywhere. In PSP you (1) **declare** the subscription in your module's `boot()`, and (2) write a **listener class**.

Step 1 — subscribe in `Provider::boot()`

```
public function boot(HttpPipeline $http, CliPipeline $cli,
                    WorkerPipeline $worker, EventBus $events): void
{
    // add_action('task.created', SendTaskEmailListener) ← the PSP equivalent
    $events->subscribe('task.created', SendTaskEmailListener::class);
}
```

Step 2 — the listener class

```
final class SendTaskEmailListener implements EventListenerContract
{
    public function __construct(private NotificationService $notifications) {}

    public function handle(IntegrationEventContract $event): void
    {
        if ($event->version() !== '1.0') return; // always version-check first

        $data = $event->payload();
        $this->notifications->notifyNewTask($data['taskId'], $data['title']);
    }
}
```

WordPress	PSP Platform
<code>do_action('task_created', \$id)</code>	<code>\$eventBus->dispatch(new TaskCreatedIntegrationEvent(...))</code>
<code>add_action('task_created', 'fn')</code>	<code>\$events->subscribe('task.created', Listener::class)</code>
callback function	a class implementing <code>EventListenerContract</code>
args are loose <code>\$variables</code>	args are a typed, versioned <code>payload()</code>
one fatal callback can white-screen the site	a failing listener is caught & logged — others still run

Crash isolation (better than WP): the `EventBus` wraps every listener in a `try/catch`. If `SendTaskEmailListener` throws, the error is logged and the *next* listener still gets the event. In WordPress, one fatal in a hooked callback can take down the whole request.

6. Going Background: Events + the Job Queue

Sometimes a listener should not make the user wait (sending email, pinging Google). The pattern is: the listener does not do the slow work itself — it **pushes a job** onto the queue, and a worker does it later. Real example from the SiteSEO plugin:

```
// When a URL is published, queue an IndexNow ping instead of blocking the request
final class EnqueueIndexNowListener implements EventListenerContract
{
    public function __construct(private QueuePort $queue, private string $queueName)
    {
    }

    public function handle(IntegrationEventContract $event): void
    {
        $url = (string)($event->payload()['url'] ?? '');
        if ($url === '') return;
        $this->queue->push('seo.indexnow', ['urls' => [$url], ...], $this->queueName);
    }
}
```

WordPress parallel: this is like using Action Scheduler / WP-Cron instead of doing slow work inside the `do_action` callback. The event fires instantly; the heavy lifting happens in a separate worker process.

7. The Golden Rules (print this page)

✓ Always	✗ Never
Dispatch integration events <i>after</i> commit	Dispatch inside a <code>try/catch</code>
Call <code>collector->discard()</code> in every rollback	Leave events buffered after a failure (phantom events)
Put a <code>version</code> on every integration event	Ship an unversioned public event
Use plain values (string/int/float) in integration events	Put <code>Money</code> / entities in an integration event
Version-check at the top of every listener	Assume the payload shape never changes
Name events in the past tense (<code>task.created</code>)	Name them as commands (<code>create.task</code>)
Let the EventBus isolate listener failures	Wrap listeners in your own try/catch
Fire domain events from the <i>Service</i> layer	Fire events from a Repository or Gateway

8. Quick Decision Guide

"Which event do I create?" Ask one question:

Does code in *another* module need to know?

- **No** (only my own module reacts, e.g. updating a summary table): use a **Domain Event**, collect it during the transaction.
- **Yes** (email, billing, search index, another team's module): use an **Integration Event**, dispatch it after commit.

The whole lifecycle on one line

`Entity records → Service collects → commit → release domain events`

then, only on success:

`Service dispatches Integration Event → EventBus → each Listener (isolated)`

If you remember nothing else: Domain Events are *private and reversible* (thrown away on rollback). Integration Events are *public and final* (only fired once the truth is committed). That single distinction is the heart of the entire PSP event system.

